

CARD AUTHENTICATION SYSTEM (CAS)

Dokumentacja Techniczna i Opis Systemu

Autorzy:	Kacper Gędek, Marcin Biliński, Filip Gądek, Szymon Dębkowski
Wersja dokumentu:	1.0.0
Data:	Czerwiec 2026 r.

1. Opis Ogólny Systemu

Card Authentication System (CAS) to zintegrowany system kontroli dostępu oparty na autoryzacji zbliżeniowych kart identyfikacyjnych (RFID / Smart Card). Architektura systemu opiera się na trójwarstwowym podziale odpowiedzialności, co zapewnia modułowość, łatwość testowania oraz potencjalne wdrożenie produkcyjne.

Adnotacja Architektoniczna

System został zaprojektowany z myślą o środowiskach akademickich i korporacyjnych, umożliwiając szybkie wdrożenie zarówno na fizycznej infrastrukturze czytników, jak i testowo w środowiskach zwirtualizowanych.

Główne Komponenty Systemu

- **serverCAS:** Centralna aplikacja serwerowa oparta na frameworku Spring Boot, odpowiedzialna za bazę danych użytkowników, logikę biznesową przyznawania dostępu oraz interfejs zarządzania.
- **readerUtils:** Biblioteka pomocnicza w języku Java stanowiąca wrapper dla standardowego API `Java Smart Card I/O`. Umożliwia niskopoziomową komunikację z czytnikami i kartami standardu MiFare oraz ELS.
- **mockClient:** Narzędzie CLI i biblioteka testowa służąca do symulacji zachowania fizycznych czytników RFID, weryfikacji połączeń sieciowych oraz testów obciążeniowych serwera bez użycia fizycznego sprzętu.

2. Komponent serverCAS

Serwer aplikacji stanowi serce systemu CAS. Przetwarza żądania autoryzacji wysyłane przez czytniki, sprawdza uprawnienia w bazie danych i zwraca jednoznaczny status autoryzacji.

Budowanie i Uruchomienie

Aplikacja serwerowa jest w pełni zkonteneryzowana. Do zbudowania obrazu OCI (Docker) oraz uruchomienia środowiska wraz z bazą danych wykorzystywany jest Gradle oraz Docker Compose:

```
./gradlew serverCAS:bootBuildImage
docker compose up
```

Dostępne Interfejsy i Endpointy

- **Panel Zarządzania (UI):** `http://{configured_url}/` – Graficzny interfejs użytkownika służący do zarządzania bazą kart, przypisywania uprawnień użytkownikom oraz monitorowania statusu czytników w czasie rzeczywistym.

- **API Walidacji:** `http://{configured_url}/validation` – Endpoint przeznaczony do komunikacji maszynowej dla czytników RFID przesyłających zapytania autoryzacyjne.

Specyfikacja API Walidacji

Zapytanie autoryzacyjne realizowane jest za pomocą metody `GET` (lub opcjonalnie `POST` w zależności od konfiguracji sieciowej) z ciałem żądania w formacie JSON.

Przykład Żądania (Request)

```
GET http://localhost:8080/validation
Content-Type: application/json

{
  "readerNumber": "1111222233334444",
  "cardNumber": "5555666677778888"
}
```

Przykład Odpowiedzi (Response)

```
Content-Type: application/json

{
  "cardOwner": "Jane Doe",
  "readerName": "Server Room",
  "validation": "ACCESS_GRANTED"
}
```

Słownik wartości pola `validation`

Wartość	Opis i zachowanie systemu
<code>ACCESS_GRANTED</code>	Dostęp przyznany. Karta jest zarejestrowana i posiada aktywne uprawnienia do danego przejścia/czytnika.
<code>ACCESS_DENIED</code>	Dostęp zabroniony. Karta istnieje w bazie, ale nie posiada uprawnień do tego konkretnego punktu dostępowego lub jest zablokowana.
<code>ERROR</code>	Błąd walidacji. Zwracany w przypadku, gdy numer karty lub numer czytnika nie zostanie odnaleziony w strukturach bazy danych.

3. Biblioteka readerUtils

Moduł `readerUtils` dedykowany jest dla systemów klienckich uruchamianych bezpośrednio na urządzeniach wyposażonych w fizyczne czytniki kart. Izoluje on dewelopera od skomplikowanego, niskopoziomowego API standardu PC/SC w Javie.

Uruchomienie Wersji Demonstracyjnej

W celu szybkiej weryfikacji poprawności działania podłączonego czytnika można uruchomić wbudowane demo:

```
./gradlew runDemo
```

Skrypt ten wylistuje wszystkie dostępne terminale i wyświetli UID pierwszej przyłożonej karty w formacie tekstowym.

Przykład Implementacji Programistycznej

```
Reader reader = new Reader();
reader.initialize();

// Pobranie listy dostępnych czytników fizycznych
System.out.println(reader.getReaders());

// Wybór pierwszego dostępnego czytnika i odczyt UID karty
reader.setReader(0);
String cardUID = reader.getUID();
System.out.println("Card UID: " + cardUID);
```

Integracja z Elektronicznymi Legitymacjami Studenckimi (ELS)

W celu przyspieszenia rejestracji i onboardingu nowych użytkowników, biblioteka posiada dedykowaną funkcję parsowania sektorów danych zabezpieczonych w Elektronicznych Legitymacjach Studenckich (ELS). Wywołanie metody `getElsData()` pozwala na natychmiastowy odczyt danych personalnych studenta.

Struktura Obiektu Danych ELS (Java)

```
public class elsData {
    private String name;
    private String surname;
    private String albumNumber;
}
```

Ważna uwaga dotycząca Javadoc

Pełna dokumentacja techniczna kodu źródłowego dla klas pomocniczych i klas sterowników może zostać wygenerowana lokalnie za pomocą polecenia: `./gradlew readerUtils:javadoc`. Wyjściowe pliki HTML zostaną zapisane w katalogu `build/docs/javadoc/index.html`.

4. Komponent mockClient

Aplikacja `mockClient` służy do celów laboratoryjnych, testowych oraz wdrożeniowych. Pozwala emulować zachowanie terminala wejściowego i wysyłać zapytania sieciowe bezpośrednio do instancji `serverCAS`.

Interfejs Linii Poleceń (CLI)

Interaktywne narzędzie tekstowe uruchamiane jest poleceniem:

```
./gradlew mockClient:run
```

Menu konsoli udostępnia intuicyjne operacje diagnostyczne:

- **Server Health Check:** Weryfikacja połączenia i statusu żywotności serwera CAS.
- **Inicjalizacja Symulatora Czytnika:** Możliwość ręcznego zdefiniowania identyfikatora punktu dostępowego lub wygenerowania losowego ID czytnika.
- **Skanowanie Karty:** Wprowadzenie konkretnego numeru karty bądź test losowy w celu natychmiastowej weryfikacji uprawnień w serwerze.
- **Statystyki:** Podgląd sumarycznej liczby wykonanych zapytań w bieżącej sesji.

Użycie Programistyczne w Testach Automatycznych

Klasa `MockClient` może zostać zintegrowana w zewnętrznych testach integracyjnych systemu:

```
// Inicjalizacja z domyślnym adresem serwera (http://localhost:8080)
MockClient client = new MockClient();

// Alternatywna inicjalizacja z niestandardowym adresem URL
MockClient customClient = new MockClient("http://localhost:9090");

// Konfiguracja wirtualnego punktu dostępowego
client.getRfidSimulator().initialize("1111222233334444", "Main Gate Reader");

// Symulacja przyłożenia karty i pobranie logu komunikacji HTTP
CommunicationLog log = client.simulateScanAndValidate("5555666677778888");

// Wyświetlenie szczegółowych logów żądania i odpowiedzi HTTP
System.out.println(log.toDebugString());
```

```
// Analiza i weryfikacja biznesowa wyniku
ValidationResult result = client.getValidationService().parseResult(log);
if (result != null) {
    System.out.println("Access validation: " + result.getValidation());
    System.out.println("Card Owner: " + result.getCardOwner());
}
```